

Práctica m43 - Modelado de Base de Datos Relacional en Entorno Fintech

Descripción del proyecto

En esta práctica desarrollo un modelo de base de datos relacional a partir de archivos CSV, integrándolos en SQLite utilizando Python. El objetivo es organizar la información en tablas relacionadas y posteriormente realizar consultas que permitan analizar los datos de forma estructurada.

Este proyecto lo planteo como parte de mi enfoque profesional dentro de mi iniciativa de trabajo en análisis de datos, donde busco aplicar buenas prácticas en el manejo, estructuración y consulta de información.

Enfoque del proyecto

Decidí trabajar con un enfoque fintech porque es un sector que me interesa especialmente. Me llama la atención la combinación entre tecnología y finanzas, así como el impacto que tiene el análisis de datos en la toma de decisiones dentro de este tipo de plataformas.

Además, me gustaría desarrollarme profesionalmente en este entorno, por lo que considero importante comenzar a trabajar con modelos que simulen escenarios reales de este tipo de industria.

Desarrollo

A lo largo del proyecto trabajo con tres entidades principales: clientes, cuentas y transacciones. A partir de estos datos, construyo una base relacional que me permite aplicar consultas SQL como JOINS, lógica condicional y subconsultas, con el fin de obtener información relevante y simular análisis que podrían realizarse en un entorno real.

Preparación del entorno

En esta sección preparo el entorno de trabajo importando las librerías que voy a utilizar.

Uso `pandas` para trabajar con los archivos CSV y manipular los datos, y `sqlite3` para gestionar la base de datos relacional donde voy a almacenar la información.

También establezco la conexión con la base de datos SQLite. Debido a que el notebook se encuentra dentro de la carpeta `notebooks`, ajusto la ruta para acceder correctamente a la ubicación donde se creará la base de datos.

```
In [2]: import pandas as pd
import sqlite3

# conexión a base de datos SQLite (ajustando ruta relativa)
conn = sqlite3.connect('../data/processed/fintech.db')
```

Carga de los datos

En este paso realizo la carga de los archivos CSV que contienen la información base del proyecto.

Debido a que el notebook se encuentra dentro de la carpeta `notebooks`, ajusto las rutas para acceder correctamente a los archivos ubicados en la carpeta `data/raw`.

Estos datos representan las entidades principales del modelo: clientes, cuentas y transacciones.

```
In [3]: clientes = pd.read_csv('../data/raw/clientes.csv')
cuentas = pd.read_csv('../data/raw/cuentas.csv')
transacciones = pd.read_csv('../data/raw/transacciones.csv')
```

Exploración inicial de los datos

Antes de integrar los datos en la base de datos, reviso su estructura para asegurarme de que las columnas y los valores sean correctos.

Esto me permite validar que la información está lista para ser utilizada.

```
In [4]: clientes.head()
```

Out[4]:

	id_cliente	nombre	edad	ciudad	fecha_registro
0	1	Ana Lopez	29	CDMX	2022-01-15
1	2	Carlos Perez	40	Guadalajara	2021-11-20
2	3	Luisa Martinez	35	Monterrey	2023-02-10
3	4	Juan Torres	50	Puebla	2020-07-05
4	5	Sofia Ramirez	27	CDMX	2023-06-01

```
In [5]: cuentas.head()
```

Out[5]:

	id_cuenta	id_cliente	tipo_cuenta	saldo
0	101	1	ahorro	15000
1	102	2	credito	-5000
2	103	3	ahorro	22000
3	104	4	credito	-12000
4	105	5	ahorro	8000

```
In [6]: transacciones.head()
```

Out[6]:

	id_transaccion	id_cuenta	fecha	monto	tipo
0	1001	101	2024-01-10	5000	deposito
1	1002	101	2024-02-15	2000	retiro
2	1003	102	2024-01-05	3000	pago
3	1004	103	2024-03-12	7000	deposito
4	1005	104	2024-02-20	4000	pago

Integración en la base de datos

En este paso inserto los datos en la base de datos SQLite.

Cada DataFrame se convierte en una tabla dentro de la base de datos, lo que me permite trabajar posteriormente con consultas SQL sobre una estructura relacional.

```
In [7]: clientes.to_sql('clientes', conn, if_exists='replace', index=False)
cuentas.to_sql('cuentas', conn, if_exists='replace', index=False)
transacciones.to_sql('transacciones', conn, if_exists='replace', index=False)
```

Out[7]: 6

Validación de los datos

Aquí realizo una consulta simple para confirmar que los datos se cargaron correctamente en la base de datos.

```
In [8]: pd.read_sql("SELECT * FROM clientes", conn)
```

Out[8]:

	id_cliente	nombre	edad	ciudad	fecha_registro
0	1	Ana Lopez	29	CDMX	2022-01-15
1	2	Carlos Perez	40	Guadalajara	2021-11-20
2	3	Luisa Martinez	35	Monterrey	2023-02-10
3	4	Juan Torres	50	Puebla	2020-07-05
4	5	Sofia Ramirez	27	CDMX	2023-06-01

Consulta con INNER JOIN

En esta consulta utilizo un INNER JOIN para combinar la información de las tablas clientes, cuentas y transacciones.

El objetivo es obtener una vista completa de las operaciones realizadas, relacionando cada transacción con el cliente correspondiente a través de su cuenta.

Este tipo de consulta me permite trabajar únicamente con los registros que tienen coincidencia en todas las tablas, lo cual es útil cuando se requiere analizar información completa y consistente.

```
In [9]: query_inner = """
SELECT
    c.nombre,
    cu.tipo_cuenta,
    t.monto,
    t.tipo,
    t.fecha
FROM clientes c
INNER JOIN cuentas cu ON c.id_cliente = cu.id_cliente
INNER JOIN transacciones t ON cu.id_cuenta = t.id_cuenta
"""

pd.read_sql(query_inner, conn)
```

Out[9]:

	nombre	tipo_cuenta	monto	tipo	fecha
0	Ana Lopez	ahorro	5000	deposito	2024-01-10
1	Ana Lopez	ahorro	2000	retiro	2024-02-15
2	Carlos Perez	credito	3000	pago	2024-01-05
3	Luisa Martinez	ahorro	7000	deposito	2024-03-12
4	Juan Torres	credito	4000	pago	2024-02-20
5	Sofia Ramirez	ahorro	1000	retiro	2024-03-01

Consulta con LEFT JOIN

En esta consulta utilizo un LEFT JOIN para obtener todos los clientes, incluyendo aquellos que no tienen transacciones registradas.

Este enfoque es útil cuando se desea identificar clientes que aún no han tenido actividad, lo cual puede ser relevante para análisis de comportamiento o estrategias de negocio.

A diferencia del INNER JOIN, aquí no se pierden registros del lado izquierdo de la relación.

```
In [10]: query_left = """
SELECT
    c.nombre,
    cu.tipo_cuenta,
    t.monto,
    t.tipo
FROM clientes c
LEFT JOIN cuentas cu ON c.id_cliente = cu.id_cliente
LEFT JOIN transacciones t ON cu.id_cuenta = t.id_cuenta
"""

pd.read_sql(query_left, conn)
```

Out[10]:

	nombre	tipo_cuenta	monto	tipo
0	Ana Lopez	ahorro	2000	retiro
1	Ana Lopez	ahorro	5000	deposito
2	Carlos Perez	credito	3000	pago
3	Luisa Martinez	ahorro	7000	deposito
4	Juan Torres	credito	4000	pago
5	Sofia Ramirez	ahorro	1000	retiro

Clasificación de cuentas mediante CASE WHEN

En esta consulta utilizo la instrucción CASE WHEN para clasificar las cuentas en función de su saldo.

El objetivo es identificar de manera sencilla el estado financiero de cada cuenta, diferenciando entre cuentas con saldo positivo, saldo bajo o saldo negativo.

Este tipo de lógica es útil en entornos financieros, ya que permite segmentar información y facilitar su interpretación para análisis posteriores.

```
In [11]: query_case = """
SELECT
    id_cuenta,
    saldo,
    CASE
        WHEN saldo > 10000 THEN 'Saldo alto'
        WHEN saldo BETWEEN 0 AND 10000 THEN 'Saldo medio'
        WHEN saldo < 0 THEN 'Saldo negativo'
    END AS clasificacion_saldo
FROM cuentas
"""

pd.read_sql(query_case, conn)
```

Out[11]:

	id_cuenta	saldo	clasificacion_saldo
0	101	15000	Saldo alto
1	102	-5000	Saldo negativo
2	103	22000	Saldo alto
3	104	-12000	Saldo negativo
4	105	8000	Saldo medio

Subconsulta tipo Semi-Join

En esta consulta utilizo una subconsulta para identificar los clientes que tienen al menos una transacción registrada.

El objetivo es filtrar únicamente aquellos clientes que presentan actividad dentro del sistema, lo cual puede ser útil para análisis de comportamiento o segmentación de usuarios activos.

```
In [12]: query_sub1 = """
SELECT nombre
FROM clientes
WHERE id_cliente IN (
    SELECT id_cliente
    FROM cuentas
    WHERE id_cuenta IN (
        SELECT id_cuenta
        FROM transacciones
    )
)
"""

pd.read_sql(query_sub1, conn)
```

Out[12]:

	nombre
0	Ana Lopez
1	Carlos Perez
2	Luisa Martinez
3	Juan Torres
4	Sofia Ramirez

Subconsulta tipo Anti-Join

En esta consulta utilizo una subconsulta para identificar los clientes que no tienen transacciones registradas.

Este tipo de análisis es útil para detectar usuarios inactivos, lo cual puede apoyar estrategias como campañas de reactivación o seguimiento comercial.

En este caso la consulta no devuelve resultados, lo cual indica que todos los clientes tienen al menos una transacción registrada. Este también es un resultado válido dentro del análisis.

```
In [13]: query_sub2 = """
SELECT nombre
FROM clientes
WHERE id_cliente NOT IN (
    SELECT id_cliente
    FROM cuentas
    WHERE id_cuenta IN (
        SELECT id_cuenta
        FROM transacciones
    )
)
"""

pd.read_sql(query_sub2, conn)
```

Out[13]:

	nombre
--	--------

Subconsulta para análisis financiero

En esta consulta utilizo una subconsulta para obtener las cuentas cuyo saldo es mayor al saldo promedio de todas las cuentas.

Este tipo de análisis permite identificar cuentas con un desempeño superior al promedio, lo cual puede ser útil para segmentación o toma de decisiones.

```
In [14]: query_sub3 = """
SELECT id_cuenta, saldo
FROM cuentas
WHERE saldo > (
    SELECT AVG(saldo)
    FROM cuentas
)
"""

pd.read_sql(query_sub3, conn)
```

Out[14]:

	id_cuenta	saldo
0	101	15000
1	103	22000
2	105	8000

Conclusión

A lo largo de esta práctica desarrollé un modelo de base de datos relacional a partir de archivos CSV, integrándolos en un entorno SQLite utilizando Python.

Durante el proceso trabajé en la estructuración de la información en tablas relacionadas, lo que me permitió entender mejor cómo se organizan los datos en un contexto real y cómo se pueden vincular entre sí para obtener información más completa.

Además, a través de las consultas realizadas, como INNER JOIN, LEFT JOIN, el uso de CASE WHEN y subconsultas, pude analizar los datos desde distintas perspectivas, aplicando lógica que permite interpretar la información de forma más clara.

Este ejercicio me permitió reforzar conceptos importantes sobre bases de datos relacionales y consultas SQL, así como entender su aplicación dentro de un entorno financiero.

Considero que este tipo de prácticas son clave para desarrollar habilidades orientadas al análisis de datos en escenarios reales.

En general, este proyecto me ayudó a mejorar mi forma de trabajar con datos estructurados, desde su carga hasta su análisis, manteniendo un enfoque ordenado y orientado a resultados.